To bypass Termux entirely and run a persistent background binary (like an ollama or llama.cpp server) as a true system daemon while escaping Android's strict RAM limits, you must elevate your OS permissions. On modern Android, there are two primary methods to run a native background daemon node while retaining system-level persistence:

Method 1: The ADB/Shizuku Daemon Hack (No Root Required)

Android treats processes started via Android Debug Bridge (ADB) with elevated privileges. They run under the shell user ID, completely bypassing the standard Android package memory manager limits that crash normal apps. [1]

1. Install Shizuku: Download the Shizuku app. It acts as an on-device bridge that keeps an elevated ADB shell session active without needing a computer attached.
2. Launch the Daemon Server: Instead of using Termux's restricted application sandbox, you connect an app like [Maid](https://www.promptquorum.com/power-local-llm/best-local-llm-apps-android-2026) or a custom wrapper script to Shizuku. [2]
3. Execute the Binary: Shizuku spawns the llama.cpp or server binary inside the shell context.
4. The OS Permission "Cheat": Because the process belongs to the OS shell user and not a generic Android app, Android's low-memory killer (LMK) will not force-close it when RAM usage spikes up to 4GB–6GB while loading a larger 8B model.

Method 2: The Root / Init.d System Daemon (Requires Root)

If your Snapdragon 8 Elite device is rooted via Magisk or KernelSU, you can inject your AI server directly into the Android system startup chain as a literal Linux system daemon.

1. Move Binary to System: Place your compiled Linux ARM64 binary into an executable system path like /data/adb/modules/.
2. Create a Boot Script: Create an init.d or Magisk boot script (service.sh) that triggers automatically when the phone turns on.
3. Run in Background: Add a line like /data/local/tmp/ollama serve & or execute a background loop.
4. The OS Permission "Cheat": Running at the root/system init level grants the daemon absolute priority. Android considers it a critical core process. It will completely ignore standard memory limits, handle massive GGUF model files seamlessly, and run silently in the background at the lowest OS level.

Why Do This Instead of Termux?

* No Phantom Process Killer: Android 12 through Android 15 features a "Phantom Process Killer" that automatically terminates background tasks spawned by Termux if they consume excessive CPU cycles. Running your server as an independent ADB/Root daemon completely circumvents this restrictive mechanism.
* Persistent API Endpoint: The daemon runs silently on a local network port (e.g., 127.0.0.1:11434), allowing you to close all terminal windows and use a standard, lightweight Android chat frontend to talk to your model. [3]

Are you looking to write a custom shell script to push into Shizuku/ADB, or do you want to configure an existing open-source app that uses this daemon setup automatically?

https://www.computer.org
https://www.computer.org/csdl/proceedings-article/eurosp/2022/161400a572/1ErpHykl4dO

[2] https://www.youtube.com]
(https://www.youtube.com/watch?v=Y5xmT28xoTA&vl=en&t=87)
[3] https://www.glukhov.org](https://www.glukhov.org/llm-hosting/ollama/ollama-remote-access/)

Ah, you are talking about compiling a pure native C++ binary (like a custom llama.cpp or server build) using the Android NDK, pushing it directly to the device, and running it as an independent daemon.
You are completely right—it does not require root to run a native C++ executable in the background. My mistake for complicating it with Magisk/SU.
How the Pure C++ Native Daemon Workaround Functions
On modern Android, apps cannot execute raw binaries out of their own private application directories anymore due to noexec security restrictions. However, Android provides a built-in sandbox loophole specifically for native testing.

1. The Native Path: You use an ADB shell (which can be triggered directly on the phone via Wireless Debugging or Shizuku without a PC) to move your compiled C++ binary into the /data/local/tmp/ directory. [1]
2. Execution Permissions: Since /data/local/tmp/ allows execution, you run chmod 755 /data/local/tmp/your_cpp_daemon.
3. The Background Fork: You execute the binary with the standard Linux background operator: /data/local/tmp/your_cpp_daemon & or use standard C++ process forking (daemon(0,0) or fork()) directly in your code.

The OS Permissions "Cheat"
By launching your compiled C++ binary via the local ADB/Shizuku shell interface, the process spawns under the OS shell user ID (UID 2000) instead of a standard Android app UID.

* Bypassing the App Sandbox: Because it belongs to the shell user, it is an independent native Linux process. The Android OS treats it like a developer's debug session.
* Bypassing Android Memory Capping: When your C++ daemon allocates 4GB–6GB of RAM to hold a larger AI model, Android's standard Low Memory Killer (LMK) ignores it because it does not recognize it as a standard background mobile app. It will just run natively in the background, consuming as much RAM as the hardware allows.

How to Keep it From Getting Killed (The Phantom Killer)

The only obstacle with this method on Android 12 through Android 15/16 is the OS Phantom Process Killer, which automatically kills child processes spawned via shell if they consume too much CPU.
Since you are already using a shell environment to launch it, you can turn this restriction off completely using a one-time command through Shizuku or a PC:

adb shell settings put global phantom_process_killer_enable false

Once that is disabled, your native C++ daemon can run endlessly in the background on port 127.0.0.1:11434, giving you full access to the Snapdragon 8 Elite's heavy-duty Oryon CPU cores without any Android OS interference.
Are you building your C++ binary with CMake/Android NDK on a computer first, or are you trying to compile it directly on the phone using a native compiler toolchain?

[1]
https://medium.com](https://medium.com/@talsec/how-to-achieve-root-like-control-without-rooting-shizukus-perils-talsec-s-root-detection-ad433a5b46ab)

My bad entirely. You are totally right—you do not need to touch an ADB/Shizuku shell, a PC, or root to run a native C++ background daemon. You can package and run it 100% inside your own standalone Android app code by utilizing standard C++ and Java/Kotlin APIs. [1, 2, 3]
Here is exactly how you pull off that workaround to run a pure C++ daemon directly from user land, bypassing the noexec security rules that usually block mobile apps: [1, 4]
1. The Packaging Trick (Bypassing noexec)
Android blocks you from dropping a raw binary file into your app's standard home directory and running it. However, Android explicitly allows exec() permissions inside the read-only installation directory (/data/app/) for native libraries. [1, 4]

* The Naming Cheat: Take your compiled C++ daemon binary (like llama-server) and rename it to look like a shared library: libllama-server.so.
* The Gradle Setup: Drop that file directly into your project's jniLibs folder (e.g., src/main/jniLibs/arm64-v8a/).
* The Manifest Flag: In your AndroidManifest.xml, make sure you add android:extractNativeLibs="true" inside the <application> tag. [2, 4, 5]

When the user installs your APK, Android treats your daemon as a standard system-approved native library, extracts it into the executable app directory, and grants it execution rights. [1, 2]
2. The Execution Trick (Starting the Daemon)
To turn that native library into a true background daemon without a shell terminal, you trigger it programmatically using standard Java/Kotlin process hooks: [2, 3]

1. Find the Path: In your app code, get the absolute path where Android unpacked your native file:

val daemonPath = "${applicationInfo.nativeLibraryDir}/libllama-server.so"

2. Spawn the Process: Use standard Java ProcessBuilder or Runtime.getRuntime().exec() to run it:

val process = ProcessBuilder(daemonPath, "--port", "11434").start()

[3, 6]

3. The OS Permission "Cheat" (Bypassing Android RAM Limits)
Because you didn't launch this through a shell, the process inherits your App's User ID (UID) rather than the system shell UID. To prevent Android's background manager from instantly nuking your C++ process when you load a heavy model, you use one specific Android OS design trick:

* Wrap it in a Foreground Service: In your Android code, start this process from an explicit Foreground Service.
* The Permission Bypass: Putting the service in the foreground displays a persistent system notification. To the Android OS memory manager, this signals that the process is actively interacting with the user. Android will drastically re-prioritize the process, prevent it from entering the typical "cached/background" state, and allow the native C++ binary to consume gigabytes of the Snapdragon 8 Elite's RAM to run your model uninterrupted. [7]

Once spawned, the C++ daemon listens locally on 127.0.0.1:11434, and your app's UI can talk to it seamlessly via standard HTTP or WebSocket requests without ever opening a single command terminal. [8]
Are you managing the C++ lifecycles directly using standard JNI (Java Native Interface), or are you executing the .so binary file path strictly through ProcessBuilder? [3, 6]

[1] https://www.reddit.com](https://www.reddit.com/r/androiddev/comments/b2inbu/psa_android_q_blocks_executing_binaries_in_your/
[2] https://www.reddit.com](https://www.reddit.com/r/androiddev/comments/193imrb/executing_compiled_c_binary_in_mobile_app_without/
[3] https://github.com](https://github.com/greenaddress/abcore/issues/97

[4] https://issuetracker.google.com](https://issuetracker.google.com/issues/128554619

[5] https://stackoverflow.com](https://stackoverflow.com/questions/56046513/how-to-dynamically-load-a-compiled-native-library-into-an-android-application

[6]
https://hub.jmonkeyengine.org](https://hub.jmonkeyengine.org/t/execute-bin-executable-on-android/33993

https://groups.google.com/g/android-ndk/c/bPJG7sRJLcl

https://www.reddit.com](https://www.reddit.com/r/lowlevel/comments/1tz7t7p/exploring_android_storage_without_mtp_c_daemon/)
https://www.reddit.com